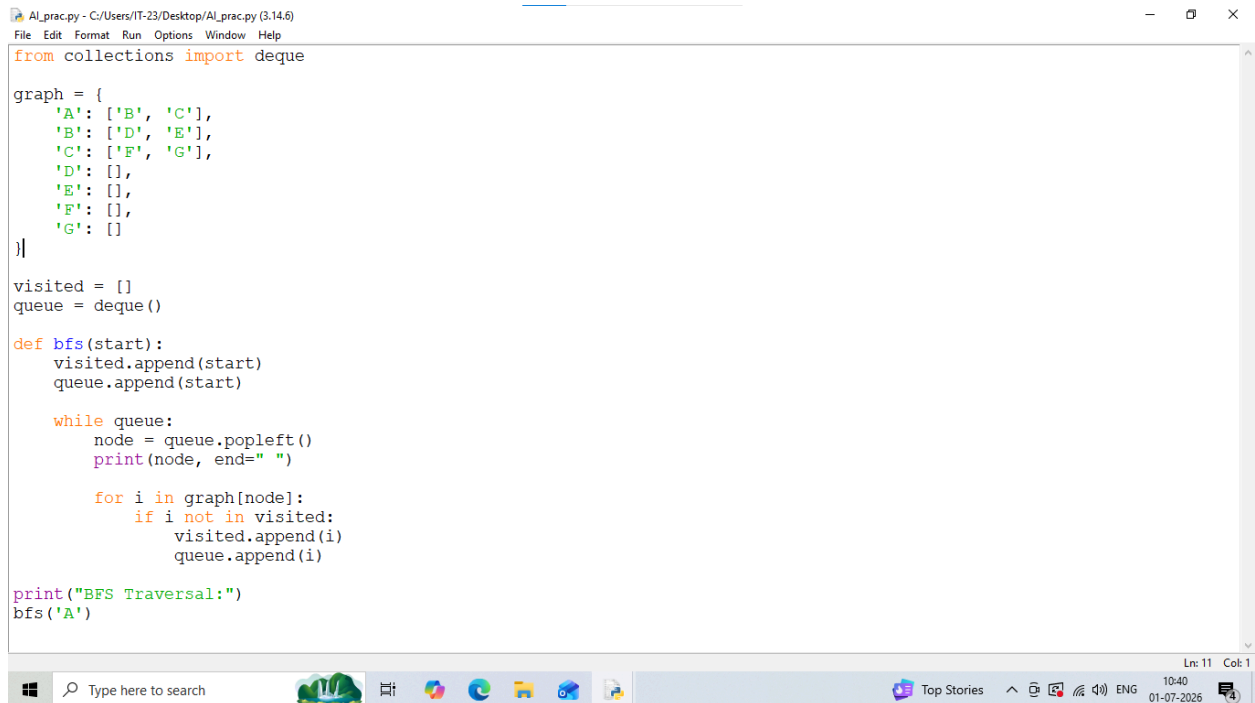


AIM : Breadth First Search & Iterative Depth First Search in Python.

- A) Implement the Breadth First Search algorithm to solve a given problem.
- B) Implement the Iterative Depth First Search algorithm to solve the same problem.
- C) Compare the performance and efficiency of both algorithms.

Breadth First Search

A screenshot of a Windows-style code editor window titled 'AI_prac.py - C:/Users/IT-23/Desktop/AI_prac.py (3.14.6)'. The editor contains a Python script for Breadth First Search (BFS). The script defines a graph with nodes A through G and their neighbors, initializes a visited list and a queue, and implements a bfs function that traverses the graph starting from node A. The Windows taskbar at the bottom shows the Start button, a search bar, and various application icons. The system tray on the right indicates the time as 10:40 on 01-07-2026.

```
AI_prac.py - C:/Users/IT-23/Desktop/AI_prac.py (3.14.6)
File Edit Format Run Options Window Help
from collections import deque

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': [],
    'E': [],
    'F': [],
    'G': []
}

visited = []
queue = deque()

def bfs(start):
    visited.append(start)
    queue.append(start)

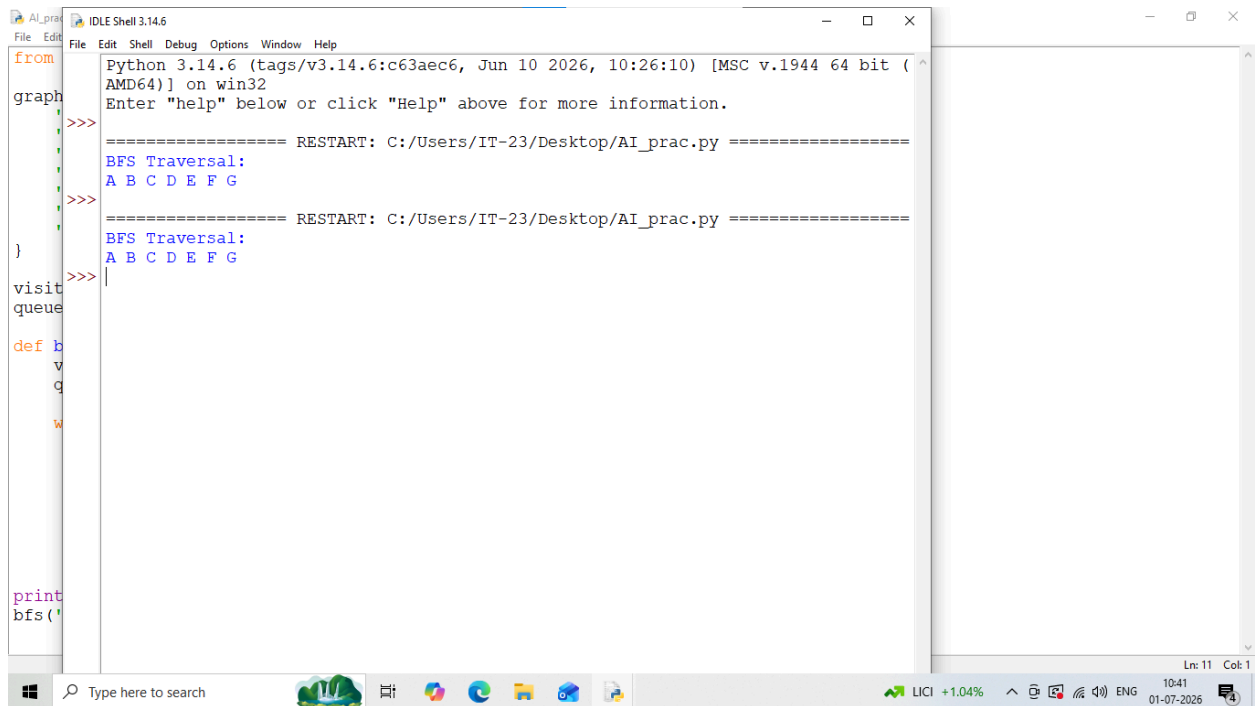
    while queue:
        node = queue.popleft()
        print(node, end=" ")

        for i in graph[node]:
            if i not in visited:
                visited.append(i)
                queue.append(i)

print("BFS Traversal:")
bfs('A')
```

SHETH L.U.J. AND SIR M.V. COLLEGE OF ARTS, SCIENCE AND COMMERCE
SUBJECT : ARTIFICIAL INTELLIGENCE

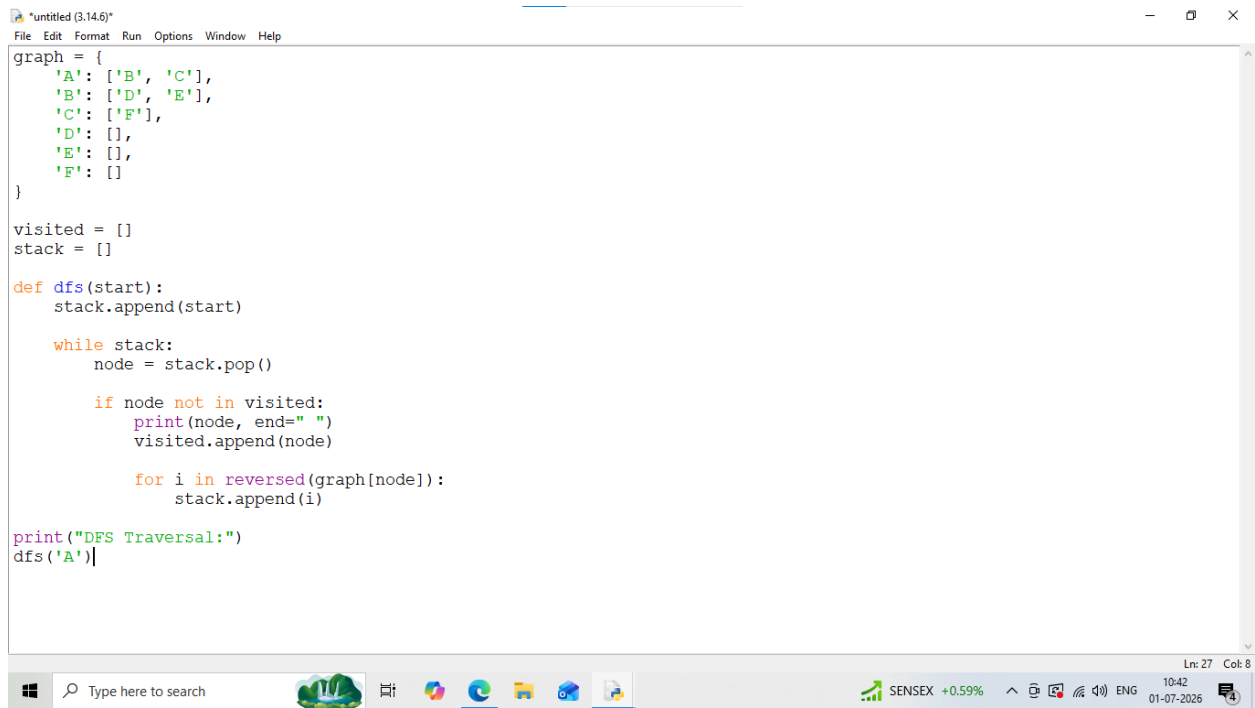
Output :



```
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>> ===== RESTART: C:/Users/IT-23/Desktop/AI_prac.py =====
BFS Traversal:
A B C D E F G
>>>
>>> ===== RESTART: C:/Users/IT-23/Desktop/AI_prac.py =====
BFS Traversal:
A B C D E F G
>>>
```

Iterative Depth First Search



```
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': [],
    'F': []
}

visited = []
stack = []

def dfs(start):
    stack.append(start)

    while stack:
        node = stack.pop()

        if node not in visited:
            print(node, end=" ")
            visited.append(node)

            for i in reversed(graph[node]):
                stack.append(i)

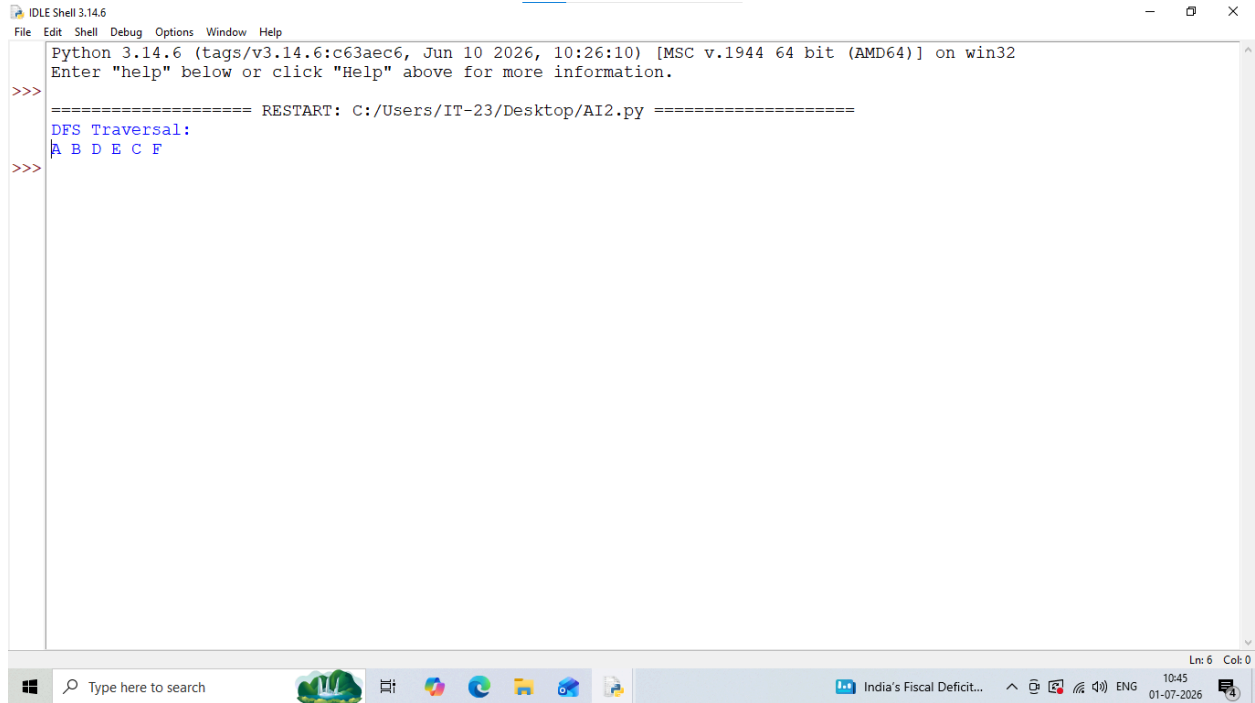
print("DFS Traversal:")
dfs('A')
```

Name : Shubham Sanjay Karape
Roll No : T085

SHETH L.U.J. AND SIR M.V. COLLEGE OF ARTS, SCIENCE AND COMMERCE

SUBJECT : ARTIFICIAL INTELLIGENCE

OUTPUT :



```
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/IT-23/Desktop/AI2.py =====
DFS Traversal:
A B D E C F
>>>
```

Compare the performance and efficiency of both algorithms.

Feature	BFS (Breadth-First Search)	DFS (Depth-First Search)
Traversal Method	Visits nodes level by level	Visits nodes as deep as possible before backtracking
Data Structure Used	Queue (FIFO)	Stack (LIFO) or Recursion
Time Complexity	$O(V + E)$	$O(V + E)$
Space Complexity	$O(V)$ (stores many nodes in the queue)	$O(V)$ (stack/recursion stores fewer nodes in many cases)
Memory Usage	Higher	Lower (generally)
Shortest Path	Finds the shortest path in an unweighted graph	Does not guarantee the shortest path
Best Use Cases	Shortest path, level-order traversal, network broadcasting	Path finding, cycle detection, topological sorting, maze solving

Name : Shubham Sanjay Karape

Roll No : T085

SHETH L.U.J. AND SIR M.V. COLLEGE OF ARTS, SCIENCE AND COMMERCE
SUBJECT : ARTIFICIAL INTELLIGENCE

Name : Shubham Sanjay Karape
Roll No : T085